

MSSQL for VS Code Completions Event Instrumentation

The MSSQL for VS Code Completions feature was built with deep event instrumentation around all core command messages. The design cleanly separates UI actions from the underlying service implementations, with each message carrying the state and configuration needed to understand, reproduce, and analyze the completion request.

All completion events are routed through a diagnostic framework that captures request parameters, local service state, model responses, telemetry, token usage, latency, and result status into a persisted session log. These logs are stored on the user's machine across VS Code sessions, making it possible to analyze both live behavior and historical trends.

A debugging UI is built directly into VS Code to make this instrumentation actionable during development. The live trace view allows developers to monitor completion events in real time, inspect request and response details, review prompt and schema context, and override feature parameters for subsequent events. This provides immediate visibility into how the feature behaves while a developer is actively testing the editor experience.

The same persisted event data is also available in a multi-session analysis view. This view aggregates events across completion sessions and groups them by configuration parameters such as model, profile, and schema budget. Developers can use this analysis to compare latency, acceptance rate, error rate, token usage, and schema-context cost across different configurations.

For deeper investigation, developers can select interesting or problematic events from historical sessions and add them to a replay trace. The replay system can resubmit the same sequence of completion events using either the original captured configuration or overridden parameters. Replay traces can also be expanded into configuration matrix runs, allowing the same event sequence to be executed across multiple profile and schema-budget combinations. Replay events are tagged in the live trace and persisted back into the session log, so experiment results can be isolated later in the aggregate analysis view.

Together, this instrumentation, live debugging experience, replay support, and configuration-matrix analysis enable several important scenarios:

1. **Deeper understanding of software behavior during testing.** Developers can directly observe completion requests, responses, local state, and telemetry while exercising features under development. They can also manipulate feature parameters and systematically compare behavior across configurations.
2. **More actionable user bug reports.** Users can provide persisted event logs when filing issues, allowing developers to inspect exact completion traces and reproduce or replay application sequences using the user's captured configuration.

3. **Targeted configuration and quality experiments.** Developers can replay the same completion scenarios across different models, profiles, and schema budgets to evaluate tradeoffs in response quality, latency, acceptance rate, and token usage.
4. **Richer runtime traces for agentic development.** The same trace data that powers the debugging experience can also be made available to coding agents, giving them concrete runtime evidence for detecting, investigating, and resolving bugs while generating application code.

The following screenshots show the end-to-end diagnostics workflow: live event capture, multi-session aggregation, replay trace construction, configuration-matrix experimentation, and replay-event analysis.

Real-time completion event tracing UI. The completions debug view captures completion events as the user types, recording them both in the UI and in a persisted session log. Developers can override feature parameters for future events, inspect event-level inputs and outputs, and view local service state through the detail panes.

The screenshot displays the Copilot Completion Debug UI. On the left, a SQL query is shown in the editor:

```
-- which queries have recently regressed
1 -- which queries have recently regressed?
2 SELECT TOP 20
3   qs.sql_handle,
4   qs.plan_handle,
5   qs.execution_count,
6   qs.total_elapsed_time / qs.execution_count AS avg_elapsed_time_us,
7   qs.total_worker_time / qs.execution_count AS avg_cpu_time_us,
8   qs.total_logical_reads / qs.execution_count AS avg_logical_reads,
9   qs.last_execution_time,
10  qs.creation_time AS plan_cached_time,
11  st.text AS query_text
12 FROM sys.dm_exec_query_stats AS qs
13 CROSS APPLY sys.dm_exec_sql_text(qs.sql_handle) AS st
14 WHERE qs.execution_count > 1
15 ORDER BY avg_elapsed_time_us DESC;
16
17
18
19 select *
20 from sys.all_columns
21 where object_id IN (SELECT object_id FROM sys.tables)
```

On the right, the Copilot Completion Debug view shows a table of events:

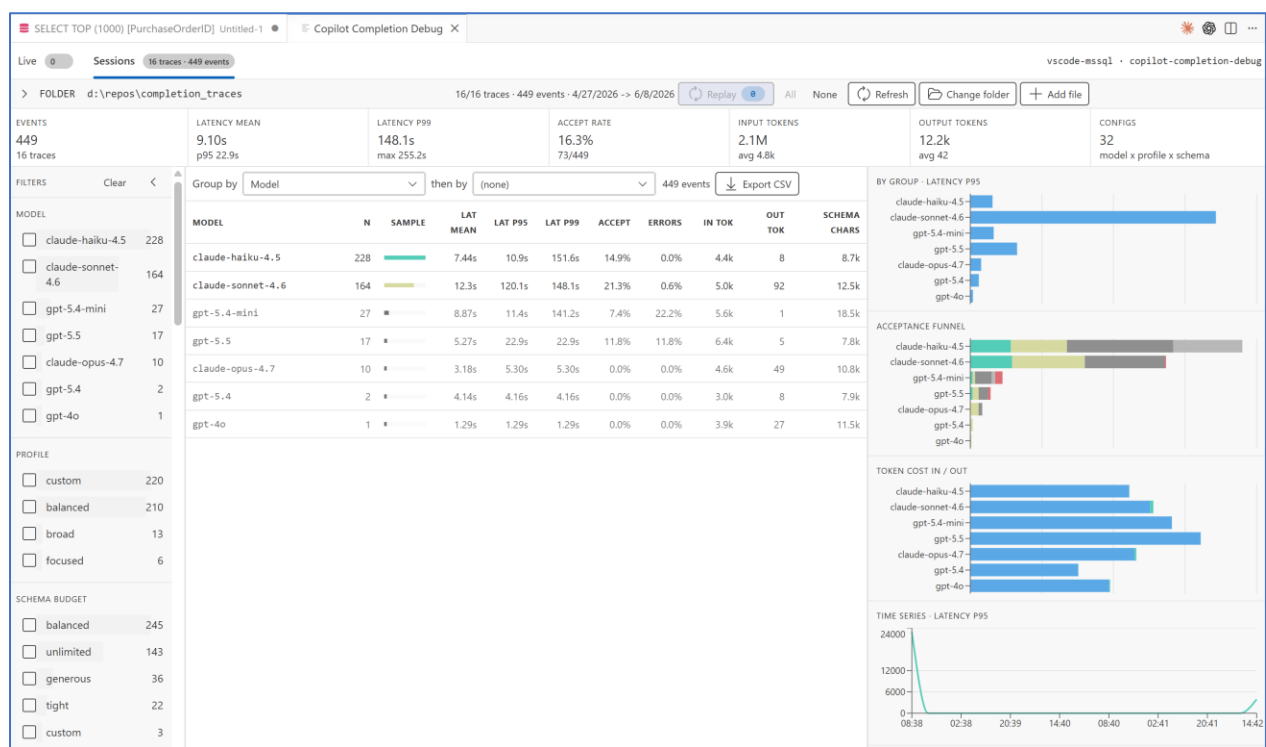
#	Time	Document	Ln:Col	Trigger	Mode	Model	Latency	In/Out	Result	Info (sanitize)
1	12:42:06.874	Untitled-1	1:41	automatic	intent	claude-sonnet-4.6	3,892 ms	3,667/--	cancelled	
2	12:42:16.687	Untitled-1	1:42	automatic	intent	claude-sonnet-4.6	3,888 ms	3,667/129	accepted	SELECT TOP 20
3	12:42:18.805	Untitled-1	15:35	automatic	continuation	claude-haiku-4.5	21 ms	8/0	skipped	Skipped: com
4	12:42:40.794	Untitled-1	20:10	automatic	continuation	claude-haiku-4.5	1,075 ms	3,776/2	success	databases
5	12:42:43.313	Untitled-1	20:21	automatic	continuation	claude-haiku-4.5	1,684 ms	3,822/--	cancelled	
6	12:42:45.483	Untitled-1	21:5	automatic	continuation	claude-haiku-4.5	46 ms	3,797/--	cancelled	object_id IN
7	12:42:47.726	Untitled-1	21:7	automatic	continuation	claude-haiku-4.5	1,571 ms	3,795/11	accepted	
8	12:42:50.783	Untitled-1	21:54	automatic	continuation	claude-haiku-4.5	1,421 ms	3,868/1	emptyFromSanitizer	

Below the table, a summary section shows the following details:

- Summary:** System Prompt, User Prompt, Raw Response, Sanitized, Schema Context, Locals Dump, Telemetry
- TOKENS:** In=3,667 | Out=129
- LM REQUEST:** sent
- SCHEMA:** TS obj| system=36 | fks=0
- USER TEXT:** -- which queries have recently regressed?
- SANITIZED RESPONSE:**

```
SELECT TOP 20
  qs.sql_handle,
  qs.plan_handle,
  qs.execution_count,
  qs.total_elapsed_time / qs.execution_count AS avg_elapsed_time_us,
  qs.total_worker_time / qs.execution_count AS avg_cpu_time_us,
  qs.total_logical_reads / qs.execution_count AS avg_logical_reads,
  qs.last_execution_time,
  qs.creation_time AS plan_cached_time,
  st.text AS query_text
FROM sys.dm_exec_query_stats AS qs
CROSS APPLY sys.dm_exec_sql_text(qs.sql_handle) AS st
WHERE qs.execution_count > 1
ORDER BY avg_elapsed_time_us DESC;
```

Multi-session completion analysis UI. The multi-session analysis view aggregates completion events across all sessions stored on the user's machine. Events can be grouped by parameters such as model, profile, and schema budget, allowing developers to compare latency, acceptance rate, token usage, and error trends across configurations.



Replay trace builder UI. Events identified in the multi-session analysis view can be added to a replay trace and resubmitted to the completion system as a sequence. For each event, developers can either reuse the original captured parameters or override them with new settings, enabling targeted testing of interesting or problematic cases discovered during real-time use.

LECT TOP (1000) [PurchaseOrderID] Untitled-1

Copilot Completion Debug

Replay Trace Builder 3 events · est. 10s sequential at avg 3s/event

Use live

Reverse order

Clear all

#	SOURCE	DOCUMENT	CONFIG (MODEL · PROFILE · SCHEMA)	STATE	PROMPT PREVIEW	ACTIONS
1	2026-06-08T19:44:25.502Z 22s ago	Untitled-1 · 1:42	Copilot — Claude Sonnet 4.6 Balanced Balanced (default)	snapshot	Shared rules: - Use only ...	Up Down
2	2026-04-27T23:02:45.367Z 15s ago	Untitled-2 · 1:44	Copilot — Claude Sonnet 4.6 Balanced Balanced (default)	snapshot	Shared rules: - Use only ...	Up Down
3	2026-04-27T22:25:54.403Z 10s ago	Untitled-2 · 8:43	Copilot — Claude Sonnet 4.6 Custom Unlimited	snapshot		Up Down

EDIT CONFIG · ROW 2

CAPTURED SNAPSHOT (READ-ONLY)

MODE

snapshot

MODEL

Captured (Copilot — Claude Sonnet 4.6)

PROFILE

Captured (Balanced)

SCHEMA

Captured (Balanced (default))

Clear override

Use live settings

Remove

Apply

```
{
  "sourceEventId": "E-2",
  "capturedAt": 1788948078485,
  "configMode": "snapshot",
  "capturedConfig": {
    "profileId": "balanced",
    "modelSelector": "copilot/claude-sonnet-4.6",
    "continuationModelSelector": null,
    "useSchemaContext": null,
    "includeSqlDiagnostics": null,
    "debounceMs": null,
    "maxTokens": null,
    "enabledCategories": null,
    "forceTokenId": null
  }
}
```

Snapshot = add-time config · Override = row config · Live = current toolbar.

Matrix...

Queue

Cancel

OK

Replay configuration matrix UI. Replay traces can be expanded into targeted configuration experiments by running the same event sequence across selected profiles and schema budgets. This enables developers to compare response quality, token usage, and latency across configurations while holding the underlying completion scenarios constant.

ELECT TOP (1000) [PurchaseOrderID] Untitled-1

Copilot Completion Debug

BackRun config matrix3 events x selected configs

PROFILES - 2 SELECTED

☒ Focused

Intent-only completions with the lowest automat...

☒ Balanced

Intent and continuation completions with a mode...

☐ Broad

Intent and continuation completions with the qu...

SCHEMAS - 2 SELECTED

☐ Tight

Smallest prompt footprint; favors the most rele...☒ Balanced (default)☒ Generous☐ Unlimited

12

41s

4

2 profiles x 2 schemas x 3 events = 12

TOTAL COMPLETIONS

EST. SEQUENTIAL

MATRIX CELLS

EXECUTION ORDER

cell 1/4 Balanced x Balanced (default) -> events 1..3
cell 2/4 Balanced x Generous -> events 1..3
cell 3/4 Focused x Balanced (default) -> events 1..3
cell 4/4 Focused x Generous -> events 1..3

Matrix runs ignore per-row cart overrides; the matrix cell profile and schema win. Resulting events are tagged with replayTraceId, replayRunId, replayMatrixCellId, and replaySourceEventId.

Run queues immediately into Live and executes sequentially.

CancelStart matrix run · 12

Replay events in the live trace view. When a replay run is queued, its events appear in the live event log and are tagged with experiment metadata. These tags make it possible to isolate replay-generated events later in the multi-session analysis view for targeted comparison and analysis.

3	12:46:51.389	Untitled-2	5:45	replay	intent	claude-sonnet-4.6	2,791 ms	2,990/19	success	Balanced x Tight · src E-60 · SELECT
4	12:47:05.907	Untitled-1	20:21	explicit	continuation	claude-sonnet-4.6	2,205 ms	3,822/12	success	WHERE object_id = OBJECT_ID('N'sys.all
5	12:49:26.401	Untitled-1	1:42	replay	intent	claude-sonnet-4.6	4,233 ms	3,667/125	success	Balanced x Balanced · src E-2 · SELEC
6	12:49:29.782	Untitled-2	1:44	replay	intent	claude-sonnet-4.6	3,044 ms	3,635/110	success	Balanced x Balanced · src E-2 · SELEC
7	12:49:35.482	Untitled-2	8:43	replay	intent	claude-sonnet-4.6	5,346 ms	883/181	success	Balanced x Balanced · src E-5 · SELEC
8	12:49:40.716	Untitled-1	1:42	replay	intent	claude-sonnet-4.6	4,891 ms	4,913/122	success	Balanced x Generous · src E-2 · SELEC
9	12:49:44.170	Untitled-2	1:44	replay	intent	claude-sonnet-4.6	3,125 ms	3,635/110	success	Balanced x Generous · src E-2 · SELEC
10	12:49:48.815	Untitled-2	8:43	replay	intent	claude-sonnet-4.6	4,311 ms	883/95	success	Balanced x Generous · src E-5 · SELEC
11	12:49:53.016	Untitled-1	1:42	replay	intent	claude-sonnet-4.6	3,848 ms	3,667/126	success	Focused x Balanced · src E-2 · SELECT
12	12:49:56.826	Untitled-2	1:44	replay	intent	claude-sonnet-4.6	3,427 ms	3,635/110	success	Focused x Balanced · src E-2 · SELECT
13	12:49:59.107	Untitled-2	8:43	replay	intent	claude-sonnet-4.6	1,931 ms	883/23	success	Focused x Balanced · src E-5 · SELECT
R-10	12:49:59.316	Untitled-1	1:42	replay	intent	claude-sonnet-4.6	1,348 ms	--/--	in flight	Focused x Generous · src E-2 · In fl
R-11	--	Untitled-2	--	replay	queued 11/12	claude-sonnet-4.6	--	--	queued	Focused x Generous · src E-2 · Shared
R-12	--	Untitled-2	--	replay	queued 12/12	claude-sonnet-4.6	--	--	queued	Focused x Generous · src E-5